HW07_ChiSquared_MCMC

April 28, 2026

1 Name: Katie

Add anything else for indentification/or however you want to label this notebook up here before submitting.

2 Submission

You do not need to write up a separate report since there will be space for you to write down thoughts/answer questions throughout this notebook. Instead, when you are finished, go to File –> Save and Export Notebook As –> LaTeX (or as pdf, but this option hasn't been working). This will convert the notebook to a .tex file which can be uploaded into Overleaf and compiled into an already formatted pdf.

3 Introduction

This homework goes over the concepts of calculating χ^2 and reduced χ^2 to see the goodness of fit of a model to provided data. We then implement an offshoot of this through Markov Chain Monte Carlo (MCMC) Simulations, which use a maximum log-likelihood to judge goodness of fit. It is important to note that maximizing log-likelihood is the same as minimizing chi-squared.

Please watch the following videos, then start filling in the below notebook cells to complete the problems.

Required videos to watch: 1. An introduction to χ^2 and reduced χ^2: https://www.youtube.com/watch?v=KNLhep3UZ68 2. An introduction to the background of MCMC Simulations: https://www.youtube.com/watch?v=nndtTssgtZE

Optional supplemental materials to read for χ^2 and MCMC: 1. https://events.mpifr-bonn.mpg.de/indico/event/30/material/slides/12.pdf - Read sections 2.3-2.4 and 3 2. https://prappleizer.github.io/Tutorials/MCMC/MCMC_Tutorial_Solution.html - Implementation of MCMC

3.1 Chi Squared

Equation for it:

$$\chi^2 = \sum_{i=1}^{N} \frac{(y_i^{OBS} - y_i^{MOD})^2}{\epsilon_{y_i}^2}$$

where y_i^{OBS} are the data/observations with error estimation ϵ_{y_i}, and y_i^{MOD} is the model function.

3.2 Reduced Chi-Squared

Another important value is the number of degrees of freedom in the model, ν, defined as

$$\nu = n - p$$

where n is the number of datapoints (measurements) and p are the number of fitted parameters.

With this, we can determine how well the model has fit the data by using the reduced χ^2_ν which is defined as the chi-square per degree of freedom:

$$\chi^2_\nu = \frac{\chi^2}{\nu}$$

To determine how well the model has fit the data, we use the reduced χ^2_ν: - $\chi^2_\nu < 1 \rightarrow$ over-fitting of the data - $\chi^2_\nu > 1 \rightarrow$ poor model fit - $\chi^2_\nu \simeq 1 \rightarrow$ good match between data and model in accordance with the data error

Taken from: https://events.mpifr-bonn.mpg.de/indico/event/30/material/slides/12.pdf (Section 2.3)

[1]:
```
#Import necessary packages
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
```

3.3 Simple power law [0.5 pts]

To get started, run the cell below. It's some fake noisy data that forms a power law to help ensure that your code is working. Feel free to change the parameters. The model for this problem is the power law which takes the functional form

$$y = Ax^k$$

where A is a constant and k is the exponent (or power law index).

The goal is to explore how χ^2 explores the parameter space for two parameters (A and k) by making a brute force χ^2 algorithm.

[2]:
```
np.random.seed(42) #just so I get the same values as I repeat the procedure as␣
↪a check to make sure things work

#power law params
A = 5
k = -1.7
x = np.linspace(1, 100, 100) #range of x values
y_true = A*x**k #eqn without noise

noise = np.random.normal(0, 0.20 * y_true) #noise

y = y_true + noise #the equation I want the model to fit to
```

```
y_err = 0.2*y_true #just some errors - they are Gaussian!
```

The first thing to do is make some functions for the model, χ^2, and χ^2_ν.

[3]:
```
# Define functions
def chi2(yobs, ymod, err):
    x = (yobs - ymod) ** 2 / err ** 2
    return np.sum(x)

def chi2nu(chi2, nu):
    return chi2 / nu
```

3.3.1 Two Parameter Fit [2 pts]

Start with the two parameter fit in which for each combination of A and k, a χ^2 and χ^2_ν is computed. Print out some of these values for a few of the iterations to see how they change. What do you notice? Hint: Define a grid of values for χ^2 to search through.

Note: This is a brute force method that explores the whole defined parameter space. It does not try to get the lowest-chi squre value.

[4]:
```
# Two param fit
ypower = lambda x: A * x ** k    #define the model

A_arr = range(1, 6)
k_arr = np.linspace(-2, 2, 5)
chi2power = np.zeros((5, 5))
chi2nupower = np.zeros((5, 5))
nu = len(x) - 1

for i in range(5):
    A = A_arr[i]
    for j in range(5):
        k = k_arr[j]
        ymod = ypower(x)
        chi2power[i, j] = chi2(y, ymod, y_err)
        chi2nupower[i, j] = chi2nu(chi2power[i, j], nu)

print(chi2power)
```

```
[[2.14743822e+03 1.44587982e+04 1.45643239e+08 1.01763422e+12
  7.83076090e+15]
 [1.84369791e+03 7.96508531e+04 5.84428120e+08 4.07067371e+12
  3.13230544e+16]
 [1.56766317e+03 1.98055049e+05 1.31635712e+09 9.15911849e+12
  7.04768806e+16]
 [1.31933401e+03 3.69671385e+05 2.34143024e+09 1.62829686e+13
  1.25292239e+17]
 [1.09871043e+03 5.94499862e+05 3.65964749e+09 2.54422239e+13
```

```
1.95769131e+17]]
```

Comments: The A values are increasing from 1 to 5 moving top to bottom and the k values are increasing from -2 to 2 from left to right.

Now, find and report the lowest value of χ^2_ν and the corresponding χ^2, A, and k. These are your best fit parameters.

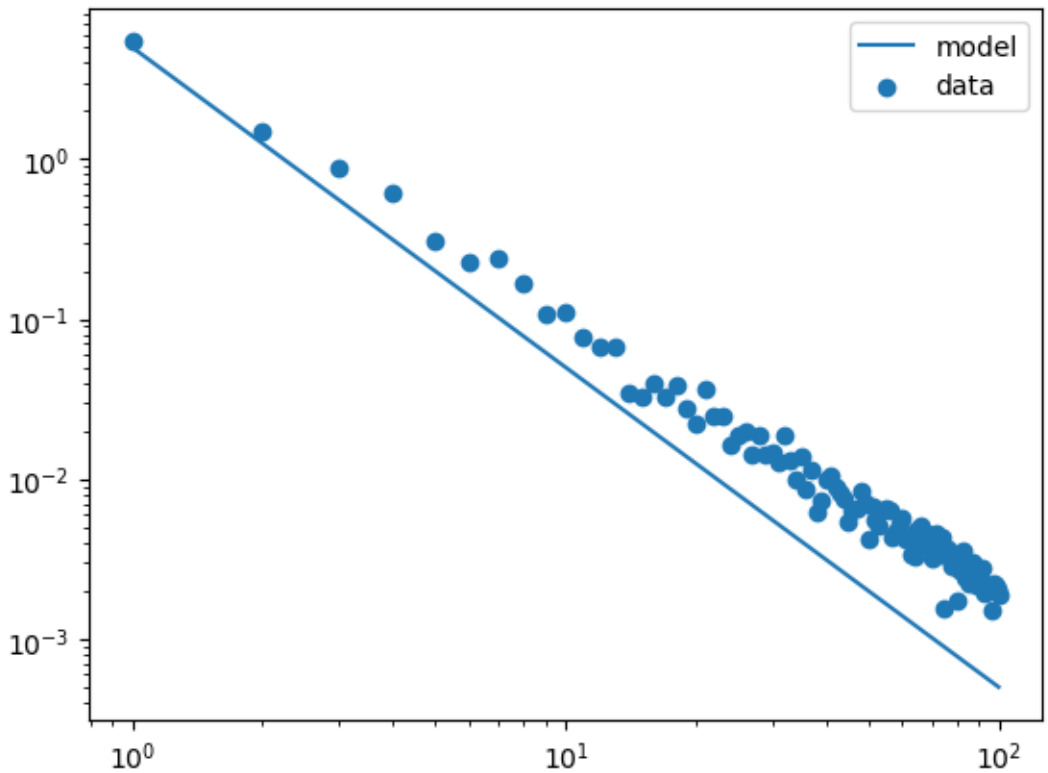
[5]:
```
# Get the best values - The printed output is fine
index = np.argmin(chi2nupower)
Aindex = (index) // 5
kindex = index % 5
print(f'The lowest \u03c7\u00b2\u1D65 is {chi2nupower[Aindex, kindex]:.2g},␣
↪with a \u03c7\u00b2 of {chi2power[Aindex, kindex]:.2g}, a A value of␣
↪{A_arr[Aindex]:.2g} and a k value of {k_arr[kindex]:.2g}.')
```

```
The lowest ²ᵥ is 11, with a ² of 1.1e+03, a A value of 5 and a k value of -2.
```

Plot the data, model with the best-fit parameters, and residuals in log-log space (You should see a line). Comment on the fit.

[6]:
```
# Plot in loglog - You should see a line
A = A_arr[Aindex]
k = k_arr[kindex]
ybest = ypower(x)

plt.loglog(x, ybest, label = 'model')
plt.scatter(x, y, label = 'data')
plt.legend()
```

[6]:
```
<matplotlib.legend.Legend at 0x1eab3ec2270>
```



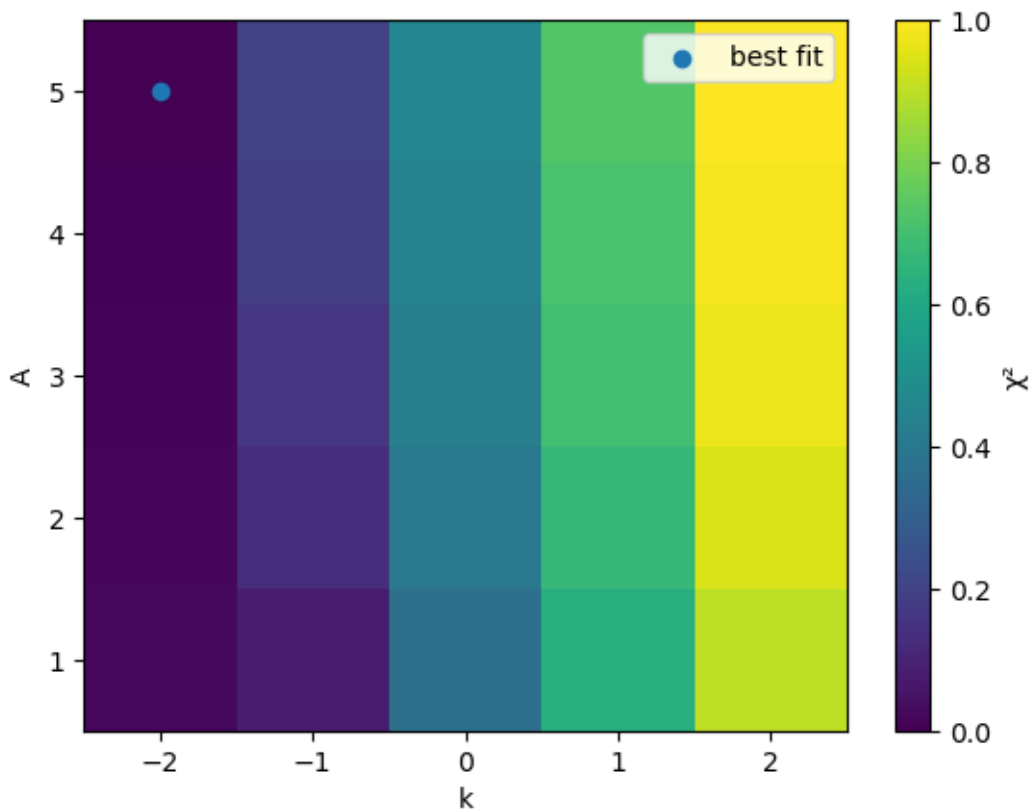
Comments: The data falls a bit above the line which has a slightly steeper slope, but the line is close to the data.

We can also see how how χ^2 changes as it cycles through the different combinations of parameters. To do this, make a contour plot of A vs k with a colorbar indicating χ^2. Include a point indicating your best fit parameters. Comment on the parameter space in relation to thest best-fit values.

Hint: Use LogNorm for the colorbar. The plot should look choppy if you have a low grid resolution, which is fine. If you want, you can make a finer grid, but that will very quickly make the χ^2 search become more computationally expensive.

[7]:
```
# Contour plot
plt.pcolormesh(k_arr, A_arr, chi2power, norm = LogNorm(np.min(chi2power), np.
 ↪max(chi2power)))
plt.scatter(k, A, label = 'best fit')
plt.xlabel('k')
plt.ylabel('A')
plt.colorbar(label = '\u03c7\u00b2')
plt.legend()
```

[7]:
```
<matplotlib.legend.Legend at 0x1eab5378690>
```



Comments: It appears that the k value had much more sway on the fit than A did in this situation with little change across columns and a lot of change across rows.

3.4 Real Data [2 pts]

Now let's apply what we've done from before to some real data. Download Stellar_Data.txt which holds mass ($M_\odot$), log stellar age (log years), log accretion rate (log $M_\odot/yr$), and log accretion rate errors. Note: The data is already in log scale, so how will that change your model function?

[8]:
```
# Load in the data
Mstar, logAge, logMacc, logMaccErr = np.loadtxt('HW07_Stellar_Data.txt',␣
↪skiprows = 1, unpack = True)
```

[9]:
```
# New model function here
ylog = lambda x: np.log(A) + k * np.log(x)
```

Use the brute force method for two parameters from before. Play around with your parameter space and print out some parameters with their χ^2, and χ^2_ν. Comment on what you notice about these values.

[21]:
```
# Brute force method
A_arr = np.linspace(.1, .5, 10)
```

```python
k_arr = np.linspace(-3.5, -2.5, 10)
chi2log = np.zeros((10, 10))
chi2nulog = np.zeros((10, 10))
nu = len(Mstar) - 1

#using same A and k values from last problem
for i in range(10):
    A = A_arr[i]
    for j in range(10):
        k = k_arr[j]
        ymodacc = ylog(logAge)
        chi2log[i, j] = chi2(logMacc, ymodacc, logMaccErr)
        chi2nulog[i, j] = chi2nu(chi2log[i, j], nu)

print(f'The \u03c7\u00b2\u1D65 values are {chi2nulog}')
print(f'The \u03c7\u00b2 values are {chi2log}')
```

```
The ² values are [[ 75.5248392   47.94436716  29.31855775  19.64741099
18.93092686
   27.16910537  44.36194652  70.50945031 105.61161673 149.66844579]
 [ 33.12126479  21.45132735  18.73605254  24.97544037  40.16949084
   64.31820395  97.42157969 139.47961807 190.49231909 250.45968275]
 [ 20.07642594  20.01360248  28.90544166  46.75194348  73.55310794
  109.30893503 154.01942477 207.68457714 270.30439214 341.87886979]
 [ 20.42691668  29.50691097  47.5415679   74.53088746 110.47486967
  155.37351451 209.22682198 272.0347921  343.79742485 424.51472024]
 [ 27.7667103   44.39054103  69.96903441 104.50219042 147.99000906
  200.43249035 261.82963427 332.18144083 411.48791003 499.74904186]
 [ 39.03795732  62.0835517   94.08380873 135.03872839 184.94831068
  243.81255562 311.63146319 388.4050334  474.13326625 568.81616173]
 [ 52.61198318  81.2482471  118.83917365 165.38476284 220.88501467
  285.33992913 358.74950623 441.11374597 532.43264835 632.70621337]
 [ 67.5536918  101.14020642 143.68138367 195.17722356 255.62772609
  325.03289126 403.39271906 490.7072095  586.97636258 692.20017829]
 [ 83.29650483 121.3246088  168.30737541 224.24480465 289.13689654
  362.98365106 445.78506822 537.54114802 638.25189045 747.91729552]
 [ 99.48330367 141.53919819 192.54975536 252.51497516 321.4348576
  399.30940268 486.1386104  581.92248075 686.66101374 800.35420937]]
The ² values are [[ 17597.28753248  11171.03754712   6831.22395635
4577.84676016
    4410.90595855   6330.40155151  10336.33353906  16428.70192119
   24607.5066979   34872.74786919]
 [  7717.25469721   4998.15927257   4365.50024251   5819.27760704
    9359.49136614  14986.14151982  22699.22806808  32498.75101093
   44384.71034835  58357.10608036]
 [  4677.80724355   4663.16937829   6734.96790761  10893.20283151
   17137.87414999  25468.98186305  35886.52597069  48390.50647291
   62980.92336971  79657.77666109]
```

```
[  4759.47158699   6875.11025638  11077.18532035  17365.69677889
  25740.64463202  36202.02887972  48749.84952201  63384.10655888
  80104.79999032  98911.92981635]
[  6469.64350007  10342.9960611   16302.78501672  24349.01036691
  34481.67211169  46700.77025105  61006.30478498  77398.2757135
  95876.68303659 116441.52675427]
[  9095.84405582  14465.46754721  21921.52743319  31464.02371375
  43092.95638889  56808.32545861  72610.1309229   90498.37278178
 110473.05103524 132534.16568328]
[ 12258.59208177  18930.84157376  27689.52746033  38534.64974148
  51466.20841722  66484.20348753  83588.63495242 102779.50281189
 124056.80706595 147420.54771458]
[ 15740.01019047  23565.66809572  33477.76239556  45476.29308997
  59561.26017897  75732.66366254  93990.5035407  114334.77981343
 136765.49248075 161282.64154265]
[ 19408.08562448  28268.6338498   39215.6184697   52249.03948418
  67368.89689325  84575.19069689 103867.92089511 125247.08748791
 148712.69047529 174264.72985726]
[ 23179.60975431  32978.63317924  44864.09299875  58835.98921283
  74894.3218215   93039.09082475 113270.29622258 135587.93801498
 159992.01620197 186482.53078354]]
```

Comments: Like before, the A values are increasing from 0.1 to 0.5 moving top to bottom and the k values are increasing from -3.5 to -2.5 from left to right. It appears that the better fit falls somewhere in the middle of the values for both k and A

Now, find and report the lowest value of χ^2_ν and the corresponding parameters and χ^2. These are your best fit parameters.

[11]:
```
# Best-fit params
indexlog = np.argmin(chi2nulog)
Aindexlog = (indexlog) // 10
kindexlog = indexlog % 10
print(f'The lowest \u03c7\u00b2\u1D65 is {chi2nulog[Aindexlog, kindexlog]:.3g},␣
↪with a \u03c7\u00b2 of {chi2log[Aindexlog, kindexlog]:.3g}, a A value of␣
↪{A_arr[Aindexlog]:.3g} and a k value of {k_arr[kindexlog]:.3g}.')
```

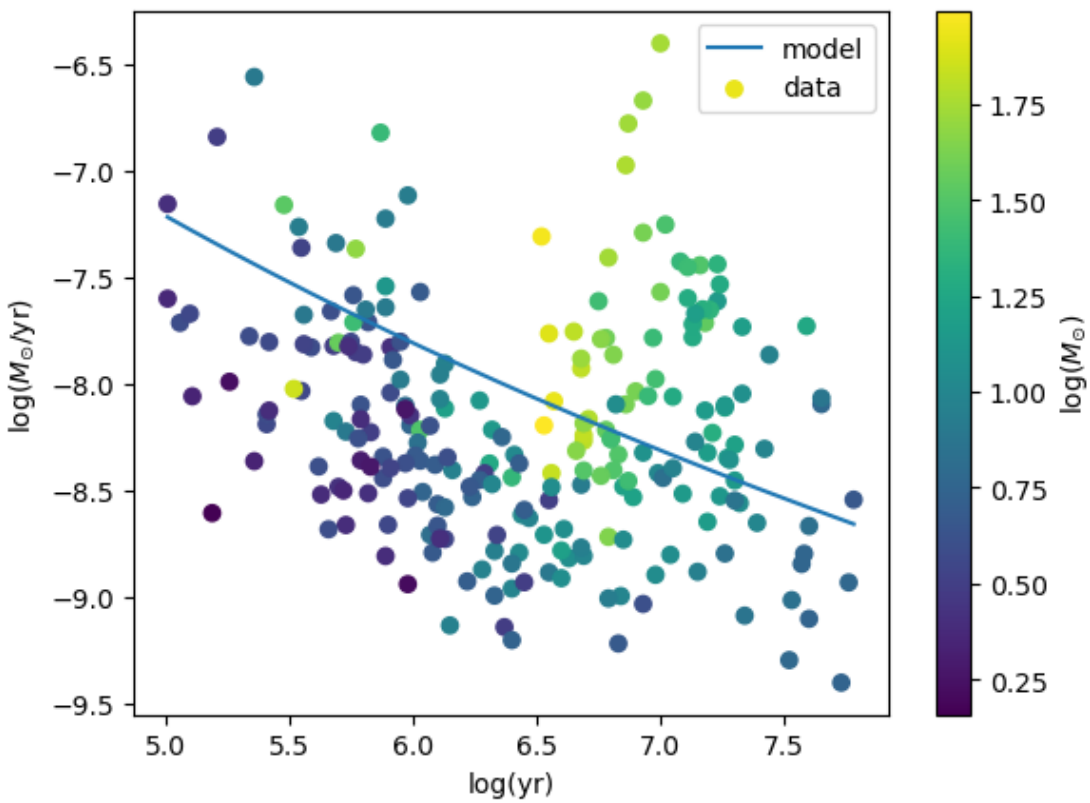
```
The lowest ²ᵥ is 18.7, with a ² of 4.37e+03, a A value of 0.144 and a k value
of -3.28.
```

As before, plot the data with errors, model with the best-fit parameters, and residuals in log-log space. Make sure to include units on the plot. Comment on the fit. Look back at χ^2_ν value you got, what does this value mean for your model? How could one improve χ^2_ν?

[12]:
```
# Plot it up
#define best fit
A = A_arr[Aindexlog]
k = k_arr[kindexlog]
ybest = ylog(logAge)
```

```
#sort the arrays to properly plot
args = np.argsort(logAge)

plt.plot(logAge[args], ybest[args], label = 'model')
plt.scatter(logAge, logMacc, label = 'data', c = Mstar )
plt.colorbar(label = r'log($M_{\odot}$)')
plt.xlabel('log(yr)')
plt.ylabel('log($M_{\odot}$/yr)')
plt.legend()
```

[12]: <matplotlib.legend.Legend at 0x1eab5584a50>



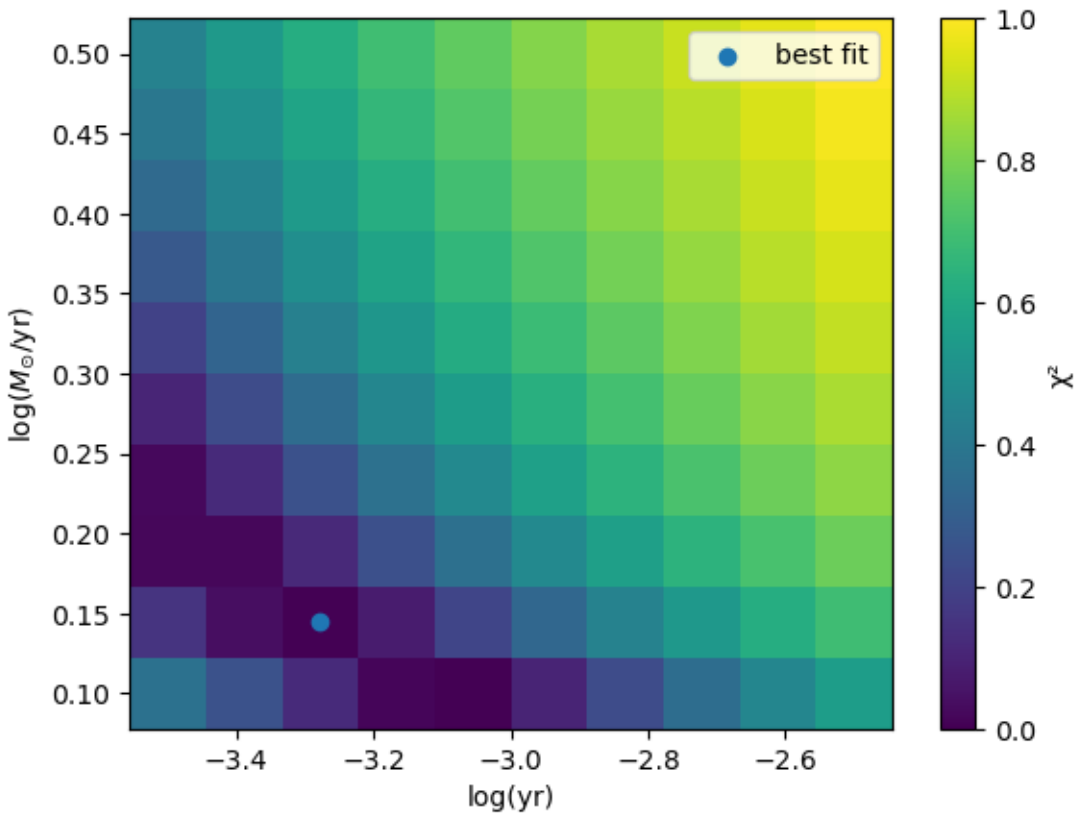
Comments: The fit appears to follow the data fairly well, but there is still a lot of scatter present.

Answers: My χ^2_ν is around 18.7. This means this function is under fit. To fix this, I could try messing with A and k more. Another solution is adding more parameters to constrain the fit in changing the number of degrees of freedom.

Also, make the contour plot of parameter 1 vs parameter 2 with a colorbar indicating χ^2 again. Add the point indicating your best-fit parameters. Comment on the parameter space in relation to these best-fit values.

[13]:
```
# Contour plot
plt.pcolormesh(k_arr, A_arr, chi2log, norm = LogNorm(np.min(chi2log), np.
 ↪max(chi2log)))
plt.scatter(k, A, label = 'best fit')
plt.xlabel('log(yr)')
plt.ylabel('log($M_{\odot}$/yr)')
plt.colorbar(label = '\u03c7\u00b2')
plt.legend()
```

[13]: <matplotlib.legend.Legend at 0x1eab567c050>



Comments: There appears to be a more diagonlal color change here. It seems that both A and k have a strong effect on the fit of this trend.

4 Fitting Stellar Accretion [5.5 pts]

In this part we are going to be using the the emcee package to run MCMC on a data set for stellar accretion https://emcee.readthedocs.io/en/stable/

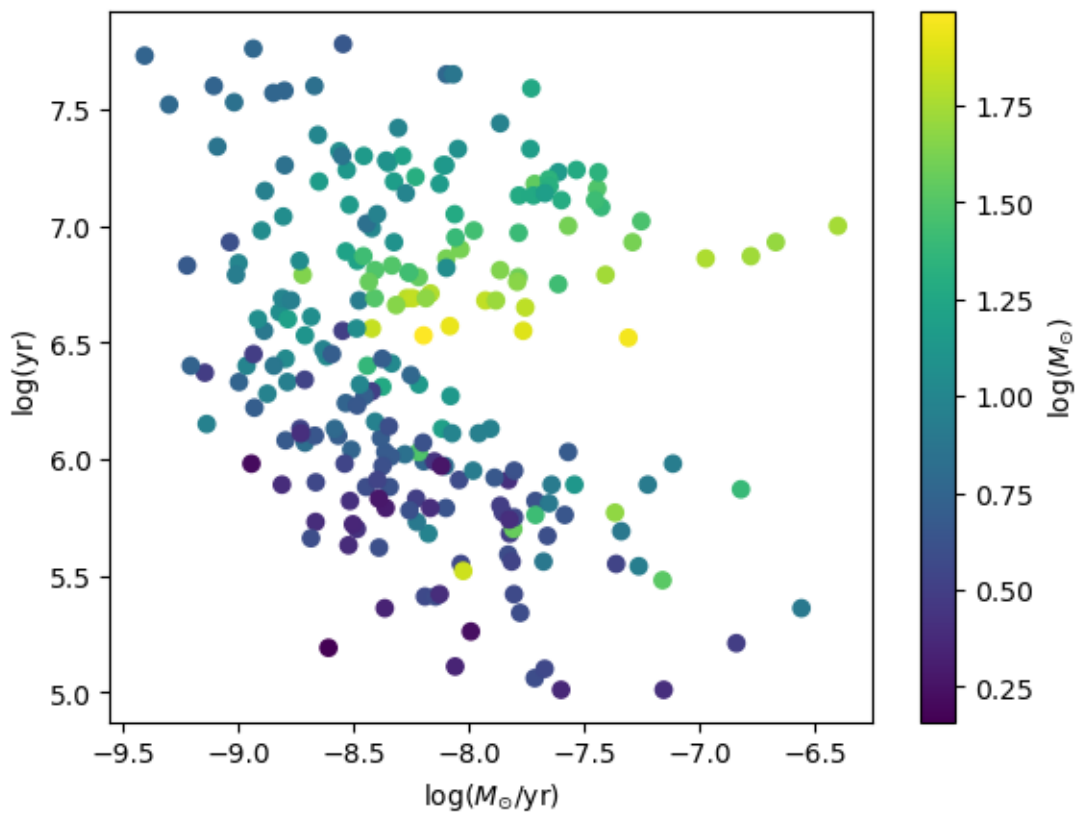
We will work through the steps in https://prappleizer.github.io/Tutorials/MCMC/MCMC_Tutorial_Solution.htm to set up the MCMC.

[17]:
```
import emcee
```

First, load Stellar_Data.txt into the notebook and make a scatter plot of logAge vs logMacc. Note: the data is already logarithmic. [0.5 pts]

[19]:
```
# Load data and plot
#data already loaded earlier
plt.scatter(logMacc, logAge, c = Mstar )
plt.colorbar(label = r'log($M_{\odot}$)')
plt.ylabel('log(yr)')
plt.xlabel('log($M_{\odot}$/yr)')
```

[19]:
```
Text(0.5, 0, 'log($M_{\\odot}$/yr)')
```



Create the linear model definition, the log likelihood definition, the lnprior definition, and the posterior probability definition. [1 pnt]

[37]:
```
# Linear Model Def
def ylin(theta, logAge = logAge):
    A, k = theta
    return np.log(A) + k * np.log(logAge)
```

```
# Log Likelihood Def
def like(theta, x, yobs, yerr):
    return -0.5 * np.sum(((yobs - ylin(theta, x)) / yerr) ** 2)

# lnprior Def
def inpr(theta):
    A, k = theta
    if 0.001 < A < 0.4 and -4 < k < -2:
        return 0.0
    return -np.inf

# Posterior Prob Def
def post(theta, x, y, yerr):
    ip = inpr(theta)
    if not np.isfinite(ip):
        return -np.inf
    return ip + like(theta, x, y, yerr)
```

Use the definitions to run the emcee. Your number of walkers should be some power of 2, and your number of iterations should be some value over 100. [1 pts]

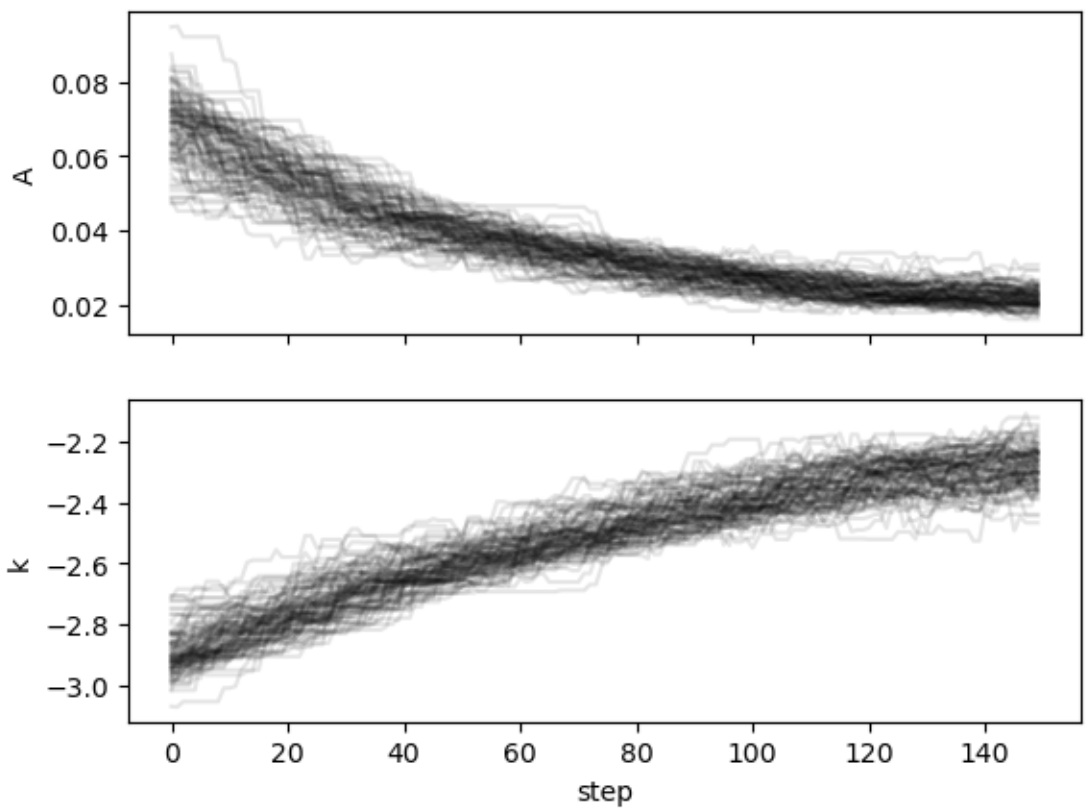
[38]:
```
# Parameters
data = (logAge, logMacc, logMaccErr)
nwalk = 100    #number of walkers
n = 150    #number of iterations
initial = np.array([A_arr[Aindexlog], k_arr[kindexlog]])
ndim = len(initial)
p0 = [np.array(initial) + 1e-7 * np.random.randn(ndim) for i in range(nwalk)]

# Run MCMC
sampler = emcee.EnsembleSampler(nwalk, ndim, post, args=data)
p0, _, _ = sampler.run_mcmc(p0, 100)
sampler.reset()
pos, prob, state = sampler.run_mcmc(p0, n)
```

Make a plot of the parameters' values over each step (typically referred to as a convergence or trace study). Comment on the value of the parameters as the MCMC runs. Hint: This information is encoded in the get_chain() function. [1 pnt]

[39]:
```
# Plot parameter values versus steps (iterations)
chain = sampler.get_chain()

fig, axes = plt.subplots(2, sharex = True)
ax = axes[0]
ax.plot(chain[:,:,0], 'k', alpha = 0.1)
ax.set_ylabel('A')

ax = axes[1]
```

```
ax.plot(chain[:,:,1], 'k', alpha = 0.1)
ax.set_ylabel('k')

axes[-1].set_xlabel('step')
```

[39]: Text(0.5, 0, 'step')



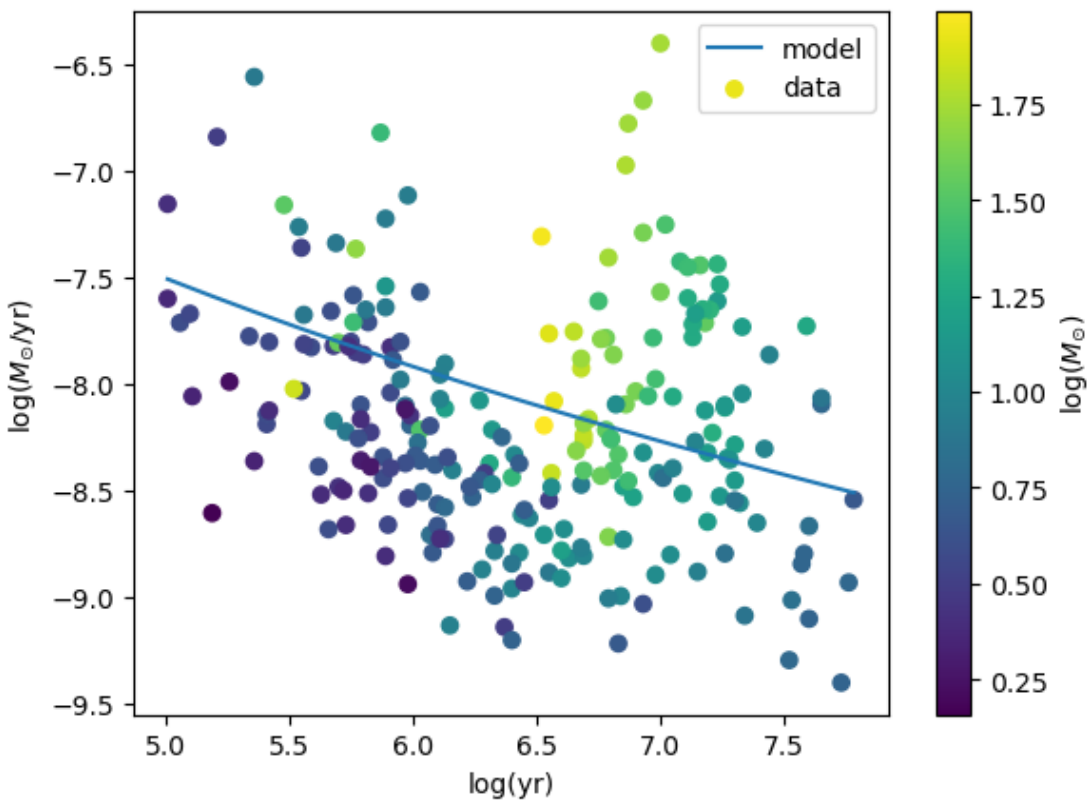
Comment: The walkers seem to have a spread among them, but they all appear to agree on a value by the end with more disagreement earlier on.

After running emcee, plot the original data with the best fit line calculated from the emcee. Comment on the plot. [1 pnt]

[40]:
```
# Plot data and best-fit line
A = np.median(chain[149,:,0])    #estimate best values
k = np.median(chain[149,:,1])

ybest = ylog(logAge)

#sort the arrays to properly plot
args = np.argsort(logAge)
```

```
plt.plot(logAge[args], ybest[args], label = 'model')
plt.scatter(logAge, logMacc, label = 'data', c = Mstar )
plt.colorbar(label = r'log($M_{\odot}$)')
plt.xlabel('log(yr)')
plt.ylabel('log($M_{\odot}$/yr)')
plt.legend()
```

[40]: <matplotlib.legend.Legend at 0x1eabaa5e0d0>



Comments: This still appears to be a good fit. The slope is shallower than the one I found in my code.

Finally, use the original data, its errors, and the emcee model data to calculate χ^2 and χ^2_v. Comment on the results. [1 pnt]

[41]:
```
#calculate chi^2 with numbers from previous cell
nu = len(Mstar) - 1

chi2emcee = chi2(logMacc, ybest, logMaccErr)
chi2nuemcee = chi2nu(chi2emcee, nu)

print(f'The \u03c7\u00b2\u1D65 value is {chi2nuemcee}')
```

```
print(f'The \u03c7\u00b2 values are {chi2emcee}')
# Calculate reduced chi-squared
```

```
The ² value is 17.32781943309487
The ² values are 4037.381927911105
```

Comments: This is still underfit, but a slightly better fit than I managed in my work.